



Khulna University of Engineering & Technology

Course No: CSE 4109

Course Title: Artificial Intelligence

Assignment No: 01

Assignment Topic: Learning Heuristics Automatically

Submitted To:

Md Mehrab Hossain Opi

Lecturer

Department of CSE

KUET

Submitted By:

Hanium Maria Joli

Roll: 2007113

Year: Fourth

Semester: First

1 Selected Research Paper

For this assignment, the paper titled “*Enhancing Dynamic Symbolic Execution by Automatically Learning Search Heuristics*” has been selected as it is directly related to my topic.

2 Problem Addressed by the Paper

Let's say we have to test a program 4,000 times before the deadline. Which execution routes needed to be investigated? Do we need to focus on just one area? Do we need to concentrate on untested code? Do we prefer more complicated or simpler paths? In current methods, these strategies are manually created by humans using experience and intuition.

3 Proposed Methodology

The researchers propose a system named **ParaDySE**, which automatically learns an effective testing strategy for each individual program. Instead of using manual programming, it actually chooses the path based on weight calculations. The steps include:

Parametric Search Heuristic:

It selects different paths for each search based on the parameters. The paper used 40 feature factors to consider. The features are given weights and then after calculation the selection happens.

. These features capture structural and semantic information about the code, such as:

- Whether a branch is located inside a loop,
- The distance of a branch from previously unexplored code,
- The depth of the execution path,
- The complexity of branch constraints.

4 Results and Findings

The key findings include:

- Automatically learned strategies consistently outperformed all human-designed heuristics across all evaluated programs.

- The approach successfully detected real bugs missed by all human strategies, including a performance bug in *gawk*.
- The learned heuristics demonstrated strong reusability, remaining effective across multiple versions of the same program over several years.

5 Connection with Course Topics

5.1 Similarities

- **Search-Based Exploration:** Both the course algorithms (e.g., A*, Greedy Best-First Search) and the selected paper perform search over a large state space. While classical AI searches through problem states (such as puzzles or graphs), dynamic symbolic execution searches through program execution paths.
- **Dependence on Heuristics:** In both cases, heuristics are essential to guide the search process. Course algorithms rely on heuristic functions to estimate the cost to reach a goal, whereas the paper uses heuristics to decide which program branch should be explored next.
- **Exponential Search Space:** Both domains suffer from exponential growth of the search space. Classical problems such as the 8-puzzle have a large but finite number of states, while real-world programs may have million and billions of states which is infeasible.
- **Scoring Mechanism:** Both approaches assign numerical scores to rank available choices. For example, A* uses $f(n) = g(n) + h(n)$ to decide next node expansion, while the paper computes weighted scores over multiple features(40) to rank execution paths.

5.2 Dissimilarities

Despite sharing common search principles, the approaches taught in the course and the selected paper has some fundamental differences.

- **Heuristic Design:** In classical AI search, heuristics are manually designed by humans (e.g., Manhattan distance, Euclidean distance), whereas the paper automatically learns heuristics using data-driven experimentation for each specific program.

- **Meaning of Heuristic:** In the course, a heuristic is an evaluation function estimating the distance to a goal state. In contrast, the paper treats a heuristic as a selection strategy that decides which execution path or branch to explore next.
- **Search Algorithm:** Course topics involve varying the search algorithm itself (A*, GBFS, RBFS), while the paper keeps the underlying algorithm fixed (dynamic symbolic execution) and optimizes only the heuristic.
- **Learning vs. Computing:** Course heuristics are computed online for each node during search. The paper learns heuristic parameters offline through training and reuses them during execution.
- **Parameterization:** Traditional heuristics are usually fixed formulas with few or no parameters, whereas the paper uses a high-dimensional parameter vector (40 feature weights) to score execution paths.
- **Evaluation Criteria:** Classical AI focuses on optimality and theoretical guarantees, while the paper evaluates success empirically using metrics such as branch coverage and bug detection rate.

6 Improvement Over Classical Methods

The ParaDySe outperformed the classical methods. The quantitative analysis includes:

- **vim-5.7 (largest benchmark):**
 - Classical best (CFDS): 7,990 branches
 - ParaDySE (learned): 8,297 branches
 - Improvement: +307 branches (+3.8%)
- **gawk-3.0.3:**
 - Classical best (CGS): 2,321 branches
 - ParaDySE: 2,684 branches
 - Improvement: +363 branches (+15.6%)
- **expat-2.1.0:**

- Classical best (CGS): 1,277 branches
 - ParaDySE: 1,327 branches
 - Improvement: +50 branches (+3.9%)
- **grep-2.2:**
 - Classical best (CGS): 1,678 branches
 - ParaDySE: 1,701 branches
 - Improvement: +23 branches (+1.4%)

The results shows that the proposed methodology improves the classical methods.

References

- [1] S. Cha, S. Hong, J. Bak, J. Kim, J. Lee, and H. Oh, “Enhancing Dynamic Symbolic Execution by Automatically Learning Search Heuristics,” *IEEE Transactions on Software Engineering*, vol. 48, no. 9, pp. 3640–3663, 2022, doi: 10.1109/TSE.2021.3101870.